

MRTIS Independent Audit #2 — the port-call assembly layer

Date	2026-08-19 17:46
Auditor	independent, adversarial, read-only
Target	HEAD = deac078 ("Close out the session: 1978-gate stays as is, no change")
Scope	git log 13937b9..HEAD — 13 commits, everything built since audit #1

Method: full rebuild of the entire chain in an isolated scratch copy; every claim re-derived from the rebuilt database in SQL rather than read out of the build's own reporting.

0. Summary

Twenty-three separate assertions were tested. The port-call assembly layer is **structurally sound and exactly reproducible** — every row count, every dollar total and every coverage percentage in the commissioning brief reproduces to the unit from a clean rebuild, and all eighteen hard guardrails were independently re-derived in SQL and confirmed rather than taken on trust.

The failures are not in the numbers. They are in **what the numbers are said to mean**: one leg-labelling rule that the code does not actually implement, one "unchanged from before" claim that is not a like-for-like comparison and hides a \$3.26M billing movement, and one comparison column that silently disagrees with the figure it is supposed to be compared against by \$2.93M.

Verdict		Count	Dollars at stake
A	Confirmed correct	14	—
B	Wrong, with evidence	8	\$6,604,500 in mis-stated or unreconciled billing
C	Unverifiable — needs a business ruling	4	routed to OPEN_QUESTIONS.md

Nothing was fixed. The real repository is byte-identical to its pre-audit state — verified below (§8).

The three findings that matter

#	Finding	Severity	Rows	Dollars
W1	54 legs report <code>activity = 'No Cargo'</code> and bill a full agency fee. The documented rule ("only a leg with nothing but layberth stops reports No Cargo") is false.	Medium	54 legs	\$413,000 billed under a "no cargo" label
W2	"\$413,000, unchanged from before this fix" is not a like-for-like comparison — leg identity changed under §8. The real §8 movement is −\$3,258,500 across 313 calls, and it is stated nowhere.	Medium	313 calls	\$3,258,500
W3	<code>port_call.agency_fee_departures_total</code> sums to \$346,692,500, not the \$349,625,500 headline. The comparison basis does not reconcile to itself, and no guardrail checks it.	Medium	388 events	\$2,933,000

1. Method and reproducibility

`scripts/`, `sql/`, `dictionaries/`, `docs/` and `data/db/mrtis.duckdb` were copied to a scratch directory; the `Zone Report*.csv` files and `fgis_source/` were symlinked read-only. All three scripts derive their paths from `__file__`, so the scratch copy is fully self-contained. The chain was run end to end:

```
build_db.py -> build_fgis_match.py -> build_port_calls.py
```

All three exited 0. The rebuild reproduced the shipped `docs/PORT_CALL_QUALITY.md` **byte-for-byte except the generation timestamp** — the build is deterministic, which is the precondition for every other check in this report meaning anything.

2. Verdict A — confirmed correct

A1. Port-call assembly is structurally sound — CONFIRMED

Table	Claimed	Rebuilt
<code>fact_zone_event</code>	290,436	290,436
<code>port_call_event</code>	290,436	290,436
<code>port_call</code>	40,170	40,170
<code>port_call_leg</code>	41,804	41,804

The spine invariant holds exactly: the symmetric difference between `fact_zone_event.event_key` and `port_call_event.event_key` is **0**, with 0 duplicates.

The guardrails were not trusted — they were re-derived. All eighteen hard checks were rewritten as independent SQL against the rebuilt database, without reference to `validate()`. All eighteen pass. Six

additional invariants the build does not check were also tested and also hold: no leg without spine events, no call without legs, `event_seq` contiguous 1..n within every call, `leg_id` consistent with `leg_seq`, `port_call.estimated_tons` equal to the sum of its legs, and `berth_stop_count` additive from legs to call.

The guardrails are genuine invariants, correctly implemented. What they do **not** cover is \$5 below.

A2. Agency fee arithmetic — CONFIRMED to the dollar

Both bases close in closed form from the tier counts, with no rounding slack:

Basis	Composition	Total	Claimed
Ruling (per leg)	$22,573 \times \$10,500 + 17,672 \times \$3,500$	\$298,868,500	\$298,868,500 ✓
Pre-ruling (per departure)	$25,863 \times \$10,500 + 22,304 \times \$3,500$	\$349,625,500	\$349,625,500 ✓

Chargeable legs **40,245** ✓. Berth departures **48,167** ✓. No leg is priced off its vessel's tier (0 exceptions).

A3. The per-departure basis is genuinely untouched by the rulings — CONFIRMED

This was the sharpest question in the brief, and the answer is yes — but it had to be proved two different ways, because two separate commits could have moved it.

\$8 (layberth): the pre-\$8 `build_port_calls.py` was checked out and the port-call layer rebuilt against the identical warehouse. Comparing the two builds call by call: **0 of 40,170 calls** changed their `agency_fee_departures_total`, and 0 changed `estimated_tons`. \$8 touches leg shape and the ruling basis only.

\$9 (register refresh): the pre-refresh `ships_register_fleet.csv` (20,101 rows) was recovered from git and `agency_fee_for()` re-evaluated for all 10,211 vessels under both registers. **0 vessels changed fee tier**. The refresh moved neither basis by one dollar. The reason is structural: `vessel_type_canonical` is priority 1 and is populated for every vessel that carries a fee at the bulk tier, so `ship_type_group` is only ever consulted for 106 departures in the whole dataset.

A4. \$8a — a layberth stop never splits a call — CONFIRMED

`split_into_legs()` treats `No Cargo` exactly like unresolved: it joins the leg in progress and never becomes `cur_activity`. Verified three ways.

- 1. Code trace.** `NON_SPLITTING = (None, "No Cargo")`; `splitting_act` is `None` for a layberth stop, so the boundary condition `splitting_act` is not `None` and `...` cannot fire on it.
- 2. Whole-dataset check.** Every leg-to-leg transition in all 1,632 split calls is `Discharge → Load` (1,617) or `Load → Discharge` (17). **No Cargo appears as a leg-boundary activity zero times.**
- 3. A real call, end to end.** Call `9757527-202606191251` is the exact sequence the claim describes:

Stop	Zone	Type	Activity	Leg
1	PCS Nitrogen	Chemical Plant	Discharge (draft_delta)	1
2	Perry Street	General Cargo — layberth	No Cargo	1
3	ADM Destrehan	Elevator	Load (dictionary)	2

The layberth stop joins the discharge leg; the split lands on the Discharge/Load boundary, exactly as if it were not there. Two legs, two fees.

The 14 ops = Layberth zones in dictionaries/zone_facility.csv were confirmed (and are the identical set to the 14 rows carrying Rule = 'No cargo ever takes place' — the two conditions in resolve_activity() are redundant but consistent).

A5. \$8b — 142 pure-layberth legs bill \$0 — CONFIRMED

Derived independently from the spine by zone name rather than from the activity column: legs whose only berth stops are layberth stops number **142**, of which **0** carry a fee. ✓

A6. \$9 register refresh — CONFIRMED, every figure

Verified directly against /Users/billy/Documents/Ships_Register/data/out/ships_register.duckdb :

Source-side claim	Verified
49,763 rows in fleet_joined	49,763 ✓
133 ship_type_group values	133 ✓
zero blank ship_type_group	0 ✓
zero null IMOs	0 ✓
zero duplicate IMOs	0 ✓

Coverage claim	Verified
99.4% of all vessels	10,148 / 10,211 = 99.4% ✓
99.8% of those with a valid IMO	10,148 / 10,172 = 99.8% ✓
port_call.dwt / .tpc on 40,055 of 40,170 (99.7%)	40,055 / 40,170 = 99.7% ✓

The 24 unmatched valid-format IMOs were verified by IMO, not by count, each one classified against the check-digit algorithm and against the source's own quarantine_pre1980 table:

- **13 checksum-invalid (MRTIS-side):** 1048576 *Wisdyln*, 1243640 *Bowling Green*, 1245184 *Ct Pilotus*, 1584932 *Andrea S*, 1622044 *John James Charles*, 6125398 *A R A Libertad*, 6665000 *Randy Anderson*, 8945819 *Sea Voyager*, 9096921 *Pis Jawa*, 9250099 *Dimitra C*, 9404765 *Aom Sophie*, 9437185 *Kristi P Smith*, 9999999 *Frenchwarship*.

- **10 behind the pre-1980 build-year gate** (all present in `quarantine_pre1980`): 5239515 *Ts Gunilla*, 6728068 *Oregon II*, 7318999 *Gyre*, 7633375 *Sunnanvik*, 7634290 *Rv Proteus*, 7710226 *Balder*, **7711725 Radcliffe R Latimer**, 8642555 *Arc Gloria*, 8642567 *J S De Elcano*, 8642799 *Esmeralda*.
- **1 genuinely absent**: 9493523 *Stena Premium* — confirmed absent from `fleet_joined` and from both quarantine tables.

13 + 10 + 1 = 24. The breakdown is exactly as claimed.

A7. Tonnage rename — CONFIRMED, both halves

Table	<code>actual_tons</code> non-null	<code>estimated_tons</code> non-null
<code>port_call</code>	0	10,545
<code>port_call_leg</code>	0	10,545
<code>port_call_event</code>	0	83,038

`actual_tons` is NULL everywhere. A grep of the whole build confirms the only writes are the literal `"actual_tons": None` — there is no code path that can populate it.

Conservation is exact, and to the *same* standard the old `actual_tons` guardrail held:

```
469,416,219 t on legs + 4,381,969 t stranded on unplaced events
                      = 473,798,188 t matched in FGIS          ✓ exact
```

A8. Claim 7 — the retraction was correct — CONFIRMED

The self-check holds, and the reasoning given for it is the right reasoning.

Radcliffe R Latimer (IMO 7711725) has `vessel_type_canonical` = 'Bulk', taken from the Zone Report's own `Type` field. `agency_fee_for()` returns `AGENCY_FEE_BULK` on its **first** branch and never reads `ship_type_group`. Evaluated directly:

Call	Result
<code>agency_fee_for('Bulk', <group>, '7711725')</code>	10,500 — what it bills
<code>agency_fee_for('Bulk', None, '7711725')</code>	10,500 — identical with no register row at all
<code>agency_fee_for(None, <group>, '7711725')</code>	3,500 — only if <code>Type</code> had been blank

The vessel has **1** berth departure and **1** chargeable leg, billing \$10,500 on both bases. **Billing impact of the backfill on this vessel: \$0.** The retraction was correct.

New detail the original retraction did not surface: 7711725 is no longer in the register at all after §9 (it was present pre-refresh as Bulk Carrier, Self-discharging, Laker). Its `ship_type` and `ship_type_group` are now both NULL. This changes nothing about the fee — see W8.

A9. Audit #1's open findings do not reach the billing layer — CONFIRMED

The Egret false merge (1782585 → 9747120) is quarantined by construction. Audit #1 found 131 workboat rows wrongly merged into tanker 9747120, fabricating \$98,000 of per-departure fee. Traced through the new layer:

Cohort	Events	Placed in a call?	Departure fee
2019 workboat rows	131	No — all unplaced	\$98,000
2020+ real tanker	32	Yes	\$14,000
2020+ real tanker	5	No	\$0

The workboat rows carry no Enter / Exit event, so the assembler cannot open a call for them and they land as unassigned . All five assembled port calls and all five legs belong to the genuine tanker , at real tanker berths (Marathon 5, MPLX Mt Airy, Exxon Baton Rouge), billing \$14,000 on the ruling basis.

The fabricated \$98,000 reaches no leg, no call, and no billable figure. It survives only inside the \$349,625,500 per-departure headline — which is one component of finding W3 below.

The ambiguous FGIS record cannot contaminate anything. It is UNMATCHED-AQUITANIA-20241013 , match_status = 'unmatched_ambiguous' , mrtis_event_key = NULL — structurally incapable of reaching a leg. (See W6: there is 1 such record, not 3.)

3. Verdict B — wrong, with evidence

W1 — 54 legs are labelled "No Cargo" and bill anyway — MEDIUM, \$413,000

The claim. sql/schema_port_call.sql and docs/PORT_CALL_QUALITY.md both state: "Real activity (Load/Discharge) always wins the leg's reported activity over No Cargo; only a leg with nothing but layberth stops reports No Cargo itself."

The first half is true. The second half is false.

build_port_calls.py:596-601 :

```
real_acts = [st for st in lg if st["activity"] and st["activity"] != "No Cargo"]
act_src   = real_acts[0] if real_acts else next((st for st in lg if st["activity"]), None)
```

real_acts requires a truthy activity. An unresolved stop has activity = None , which is falsy — so it cannot enter real_acts , and the fallback picks the No Cargo stop. Meanwhile the fee test on the next line uses a different predicate entirely:

```
real_berth = any(st["activity"] != "No Cargo" for st in lg) # None != "No Cargo" -> True
```

None *does* satisfy `real_berth`. So a leg holding one layberth stop and one unresolved non-layberth stop is labelled **No Cargo** and billed in full.

Measured on the rebuilt data:

	Legs	Fee
Legs with <code>activity = 'No Cargo'</code>	196	\$413,000
...of which contain a genuine non-layberth berth stop	54	\$413,000
...of which are pure layberth (correctly \$0)	142	\$0

Every dollar billed under a **No Cargo label — all \$413,000 — sits on a leg the documentation says cannot exist.**

The berths involved are real working berths: Chalmette Slip (7 legs), IMC Faustina (4), 7th Street (4), Arabi Terminal (3), IMTT St Rose 14, Blackwater Harvey, Phillips 66 Alliance, Globalplex.

Why it matters beyond the label. Any downstream filter written against the documented behaviour — `WHERE activity <> 'No Cargo'` to isolate revenue work, or `WHERE activity = 'No Cargo'` to find unbilled lay-ups — returns the wrong answer on all 54. The label and the money disagree, and the schema comment promises they cannot.

Two related inconsistencies fall out of the same code path:

- **7 No Cargo legs assert a `cargo_group`.** `cargo_group` is read from `lg[0]["zinfo"]` — the leg's *first* stop — while `activity` is read from a possibly *different* stop. Result: legs labelled **No Cargo** carrying `cargo_group = 'Liquid Bulk'` (IMC Faustina ×3, IMTT St Rose 14, LBC Sunshine, PCS Nitrogen) and one carrying `'Passengers'` (Julia St, a cruise berth). The `no cargo` is asserted without a source guardrail passes, because `cargo_source` is populated — it checks provenance, not coherence.
- **`first_berth_zone`, `facility_type` and `first_berth_facility` likewise describe `lg[0]`**, not the stop that generated the fee. On 20 of the 54 the leg reports the layberth (Buck Kreihs, Perry Street, Poland St, Violet Dock 2, Alabo St) as its berth while billing for work done elsewhere.

Not fixed. Recommended framing: either an unresolved stop should outrank **No Cargo** for the label (making these legs `NULL` /unresolved, which is honest), or the leg needs a separate `billable_activity` column. That is a business decision — routed to `OPEN_QUESTIONS.md`.

W2 — "unchanged from before this fix" is not a like-for-like comparison — MEDIUM, \$3,258,500

The claim (`docs/PORT_CALL_QUALITY.md`:86): the 54 legs *"bill exactly as [they] did before this ruling; only the layberth stop itself is fee-exempt."*

This cannot be true as stated, because §8 changed which stops are in which leg. The pre-§8 `build_port_calls.py` was checked out and the layer rebuilt. The leg population is not the same population:

	Pre-§8	Post-§8
Legs (total)	41,985	41,804
Split calls	1,787	1,632
Legs touching a layberth	442	421
— pure layberth	300 (all charged, \$3,017,000)	142 (\$0)
— layberth + other berth	142 (\$980,000)	279 (\$2,404,500)
Legs labelled No Cargo	402	196
No Cargo legs billing	\$3,717,000	\$413,000

The "54 legs billing \$413,000" cohort **did not exist before the fix**. There were 142 mixed legs billing \$980,000; §8 merged 158 previously-separate layberth legs into their neighbours, producing 279 mixed legs. Comparing \$413,000 to itself across that boundary is comparing two different sets of rows.

The figure that is actually true, and is stated nowhere:

§8 reduced billing by \$3,258,500 — from \$302,127,000 to \$298,868,500 — across 313 port calls.

Per-call movement: 284 calls −\$10,500, 19 calls −\$3,500, 10 calls −\$21,000. That is **1.08% of the pre-fix ruling basis**, and it is the number a reviewer of the §8 ruling needs in order to sanity-check it.

To be clear about what *is* sound: the reduction is correct and intended — 142 pure-layberth legs correctly stopped billing, and 181 legs correctly ceased to exist as separate legs. The defect is in the reporting, not the arithmetic.

W3 — the comparison basis does not reconcile to itself — MEDIUM, \$2,933,000

`port_call.agency_fee_departures_total` exists, per the schema comment, so the two bases "*can be compared directly rather than one silently replacing the other.*" It does not sum to the pre-ruling basis:

Source	Total
<code>fact_zone_event.agency_fee</code> (the true pre-ruling basis)	\$349,625,500
<code>port_call_event.agency_fee</code> (the spine — guardrailed)	\$349,625,500 ✓
<code>SUM(port_call.agency_fee_departures_total)</code>	\$346,692,500
<code>SUM(port_call_leg.agency_fee_departures)</code>	\$346,692,500
Unreconciled gap	\$2,933,000

The gap is **388 fee-bearing events on the 17,100 unplaced events**, which reach no call and so are absent from the aggregate columns:

Unassigned reason	Events	Departure fee
before_first_entry	6,579	\$2,023,000
no_open_call	10,521	\$910,000

Consequence. The over-billing ratio depends on which denominator you pick: against `fact_zone_event` it is **17.0%**; against `agency_fee_departures_total` it is 16.0%. Two analysts comparing "the two bases" as the schema invites will get different answers.

Notably, \$98,000 of this \$2,933,000 is audit #1's fabricated Egret fee (\$A9) — the gap is not purely a source-coverage artefact; it also carries known bad data.

The guardrail gap is the real issue. `agency_fees` are carried through `unchanged` compares `fact_zone_event` to `port_call_event` — both of which include unplaced events — so it passes. Nothing anywhere checks the aggregate columns. See §5.

W4 — the `ship_type_group` backfill does not implement its stated condition — LOW (latent), \$0 today

The claim: "`ship_type_group` is backfilled from `ship_type` wherever the register has no size vocabulary for that family."

The code has no such condition. `build_db.py:266` is an unconditional row-level fill:

```
reg["ship_type_group"] = reg["ship_type_group"].fillna(reg["ship_type"])
```

Nothing inspects whether the family has a size vocabulary. The stated guard is a description of the data at the time of writing, not of the code.

Against the live register it is dead code. The refreshed `ships_register_fleet.csv` has **0 blank `ship_type_group`** values, so the `fillna` matches nothing. Verified.

Against the register it was written for, it happened to be right. The pre-§9 register had 1,349 blanks, and **all 1,349** belonged to families with no size vocabulary anywhere in the file — Aggregates Carrier (764), Cement Carrier (445), Bulk Carrier Self-discharging Laker (57), Limestone Carrier (46), and five smaller families. **0 illegitimate fires.** The claim's *effect* was correct; the claim's *mechanism* was not.

Why this is worth recording rather than dismissing. The backfill writes a raw family name into a column that `agency_fee_for()` pattern-matches:

```
return (AGENCY_FEE_BULK if ship_type_group.startswith("Bulk Carrier") else AGENCY_FEE_OTHER)
```

'Bulk Carrier, Self-discharging, Laker' **starts with "Bulk Carrier"**. Under the old register, the backfill therefore promoted 57 Laker rows to the \$10,500 tier by string prefix — correct by luck, not by design. A future

register revision that reintroduces blanks in a family whose name happens to begin "Bulk Carrier" will silently move vessels to the higher tier at **\$7,000 per leg**, with no guardrail to catch it.

Live exposure today: \$0. The 20 vessels whose `ship_type_group` equals their bare `ship_type` and whose `Type` is blank are all tugs, OSVs, research and pilot vessels (`Yacht` , `Tug` , `Platform Supply Ship` , `Anchor Handling Vessel` , ...). None matches `Bulk Carrier*` ; all price at \$3,500, which is what they would price at with no register row at all.

W5 — the schema comment understates the over-bill — LOW (documentation)

`sql/schema_port_call.sql` on `port_call_event.agency_fee` : *"Summing this column over-bills by ~12%."*

Measured: **17.0%** (\$349,625,500 vs \$298,868,500 = \$50,757,000).

A reader using ~12% to sanity-check a figure will accept a number that is \$15M wrong. Trivial to correct; consequential if trusted.

W6 — "3 ambiguous FGIS records" is 1 record — LOW (fact correction)

The brief carries forward "the 3 ambiguous FGIS records" from audit #1. There is exactly **1**: `UNMATCHED-AQUITANIA-20241013` . It spans **3 certificate lines** (`line_count` = 3), which is where the 3 comes from. Audit #1 itself said "1 ambiguous". Full breakdown:

match_status	Records
matched	12,445
unmatched_outside_coverage	1,991
unmatched_no_candidate	91
unmatched_ambiguous	1

W7 — "99.7% populated" conceals `tpc = 0` on 10% of calls — LOW-MEDIUM (data quality)

`dwt / tpc` being *non-null* on 40,055 calls is true, but non-null is not usable:

Measure	Calls	%
<code>tpc IS NOT NULL</code>	40,055	99.7%
<code>tpc > 0</code>	36,010	89.6%
<code>tpc = 0</code>	4,045	10.1%
<code>dwt = 0</code>	49	0.1%

`tpc = 0` is a missing value encoded as a number, and it lands on **2,582 Bulk calls** — the vessels for which tonnes-per-centimetre actually matters — plus 1,031 Passenger, 116 Container, 99 Gas. 1,130 distinct vessels are affected.

Any draft-survey or deadweight calculation dividing by `tpc` will produce a division-by-zero or an infinite tonnage on 10% of calls. The headline coverage figure gives no warning. (`docs/SESSION_LOG.md` already flags TPC provenance as needing a ruling; this is a distinct, mechanical defect — zero-as-null.)

W8 — \$9 silently lost 2 register matches — LOW (informational), \$0

The refresh is a large net gain (60.9% → 99.4%), but it is not monotonic. Two vessels matched under the old register and are unmatched now:

IMO	Name	Old <code>ship_type</code>	Now
7711725	Radcliffe R Latimer	Bulk Carrier, Self-discharging, Laker	unmatched
7633375	Sunnanvik	(present)	unmatched

Both are behind the pre-1980 build-year gate that commit `deac078` resolved to leave in place, so this is a known and accepted consequence — but it was recorded as a pure gain. Both have `vessel_type_canonical = 'Bulk'` from the Zone Report, so **billing impact is \$0**; they lose only `dwt / tpc`.

4. Verdict C — unverifiable without a business ruling

Routed to `docs/OPEN_QUESTIONS.md` §11. Not decided here.

#	Question	Why the audit cannot settle it
C1	Should a leg holding a layberth stop and an unresolved working berth report <code>No Cargo</code> , report unresolved, or carry a separate billable-activity column?	W1 proves the label contradicts the fee. Which one is wrong is a definition question, not a data question. \$413,000 / 54 legs.
C2	Should <code>port_call.agency_fee_departures_total</code> include unplaced events?	W3's \$2.93M gap is defensible as "a call-level column can only hold call-level fees" — but then the two bases are not directly comparable and the schema comment should say so.
C3	Should <code>tpc = 0</code> be stored as NULL?	W7. Zero is a legitimate value for a barge and a missing value for a Capesize; only the register owner can say which the 4,045 are.
C4	Should the 2 berthed legs with no fee bill?	Both vessels have no usable IMO and no type from either source, so <code>agency_fee_for()</code> returns <code>None</code> by design. Both did berth. Fee at stake ≤ \$21,000.

5. Guardrail coverage — what the checks do not check

The eighteen hard guardrails are correct and were independently confirmed. They are also **structural only**: every one asserts a shape (uniqueness, referential integrity, set equality, ordering, sum-of-parts). Not one

asserts a *value* is right. The gaps below are all currently clean — which is exactly why they are worth naming, since nothing would catch them going wrong.

Gap	Today	Would catch
No check that <code>actual_tons</code> is NULL — the central invariant of the whole §5 rename	0 non-null ✓	any future source wiring <code>actual_tons</code> by accident
No check that a berthed, real-work leg <i>does</i> bill — the fee guardrail is one-directional (never berthed ⇒ no fee)	2 legs, both legitimately unbillable	a regression silently zeroing fees on thousands of legs
No check that a fee matches its vessel's tier	0 mispriced ✓	a tier-table or <code>agency_fee_for()</code> regression
No reconciliation of <code>agency_fee_departures_total</code> to <code>fact_zone_event</code>	\$2,933,000 adrift — W3	the live defect
No coherence check between <code>activity</code> and <code>cargo_group</code>	7 contradictory legs — W1	the live defect
No check that leg attributes describe the stop that generated the fee	20 of 54 legs report the wrong berth — W1	the live defect

Two of the six are already failing silently. `no cargo` is asserted without a `source` illustrates the pattern precisely: it verifies that `cargo_source` is populated, which is a provenance check, and passes cleanly on a leg asserting `Liquid Bulk` cargo alongside `activity = 'No Cargo'`.

6. What is genuinely well built

Worth stating plainly, because the findings above are all narrow:

- **The spine discipline works.** One row per source event, always, with unplaced events kept and reason-coded rather than dropped. This is what let the Egret false merge be *contained* rather than propagated — the workboat rows had nowhere to go, and the billing layer never saw them.
 - **The build is deterministic.** A cold rebuild reproduced the shipped quality report byte-for-byte. Almost nothing else in this report would be checkable otherwise.
 - **The rulings are implemented as ruled.** §8a's split behaviour is exactly right, verified on a real Discharge → layberth → Load call. §9's coverage figures are exact. The tonnage rename is complete and conserving.
 - **Evidence is ranked, not averaged.** `resolve_activity()` names its method on every row and flags conflicts (851 legs) instead of silently resolving them.
 - **The pre-ruling basis really is preserved.** Both rulings were tested by rebuild against their own predecessors, and neither moved it by a dollar.
-

7. Findings by severity

ID	Finding	Severity	Rows	Dollars
W1	54 legs labelled <code>No Cargo</code> bill in full; documented rule is false	Medium	54 legs (+7 cargo, +20 berth)	\$413,000
W2	"unchanged from before" is not like-for-like; §8's real impact unstated	Medium	313 calls	\$3,258,500
W3	<code>agency_fee_departures_total</code> does not reconcile; unguarded	Medium	388 events	\$2,933,000
W7	<code>tpc = 0</code> on 10.1% of calls behind a "99.7% populated" headline	Low-Med	4,045 calls	—
W4	Backfill lacks its stated guard; latent tier promotion	Low (latent)	0 today	\$7,000/leg if it fires
W5	Schema comment says ~12% over-bill; actual 17.0%	Low	—	\$15M mis-estimate if trusted
W6	"3 ambiguous FGIS records" is 1 record / 3 lines	Low	1 record	—
W8	2 register matches lost in §9, recorded as pure gain	Low	2 vessels	\$0

Total mis-stated or unreconciled: \$6,604,500. No finding indicates money is billed at the wrong *rate*; every dollar figure above is a labelling, reconciliation or reporting defect.

8. Repository integrity

Per William's standing practice, all work was done in a scratch copy. The real repository was verified byte-identical afterwards:

- SHA-256 manifest of **74 files** across `scripts/ sql/ dictionaries/ docs/ data/ fgis_source/` plus all 36 `Zone Report*.csv`, taken before and after: **identical, zero differences.**
- `git status --porcelain` before and after: **identical** (the same 5 untracked `sample_port_calls*.csv` files, unmodified).
- `data/db/mrtis.duckdb` unchanged.

Nothing in `/Users/billy/Documents/MRTIS` was modified, and nothing was fixed, other than the addition of this report and the questions appended to `docs/OPEN_QUESTIONS.md` §11.